



Create a GUI in PowerShell

Module 10

Learnings covered in this Unit

- Windows Presentation Foundation Framework Introduction
- Introduction to Visual Studio
- Graphical User Interface Design
- Advanced GUI Design
- Code Behind a Form
- Advanced Techniques

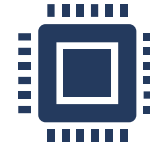
Windows Presentation Foundation Introduction

Windows Presentation Framework



**Available on all
versions of windows
with .NET 3+**

Now open source (since 2018)
<https://github.com/dotnet/wpf>



**WPF object are
created using XAML**

XAML is an XML-based Mark-up language
This is used in C# with Visual Studio for
Universal Windows Application GUIs



**Utilizes DirectX
instead of core
windows components
for better
performance**



**WPF provides a
standard set of
editable GUI features**

Why Use WPF in PowerShell



Turn your tool into something more user friendly



Gather input from a user with tools such as:

- Text boxes
- Date pickers
- Submit Buttons



Display large, interactive data sets for output

- GUI can even adapt to screen resolutions and touch screens



WPF data binding is a feature that allows us to show live data changes in the GUI

Pros of Creating WPF Objects Directly in PowerShell



Readability

Integrating the user interface (UI) directly within a PowerShell script



Dynamic UI Generation

PowerShell scripting allows for the creation of UI at runtime, and can adapt on specific conditions



Reduce External Dependencies

Combining both the UI and logic within the script reduces reliance on external files



Immediate Feedback

Developers can quickly test and adjust the UI within the script

Cons of Creating WPF Objects Directly in PowerShell



Code Maintenance

The code becomes messy and more challenging to maintain.



Rising Complexity

Creating larger GUIs manually can be challenging



Lack of Separation of Concerns

Combining UI and script logic violates the separation of concerns principal



Limited Design Tool Integration

Although tools such as Visual Studio offer strong designers for WPF, crafting these objects manually remains complex.

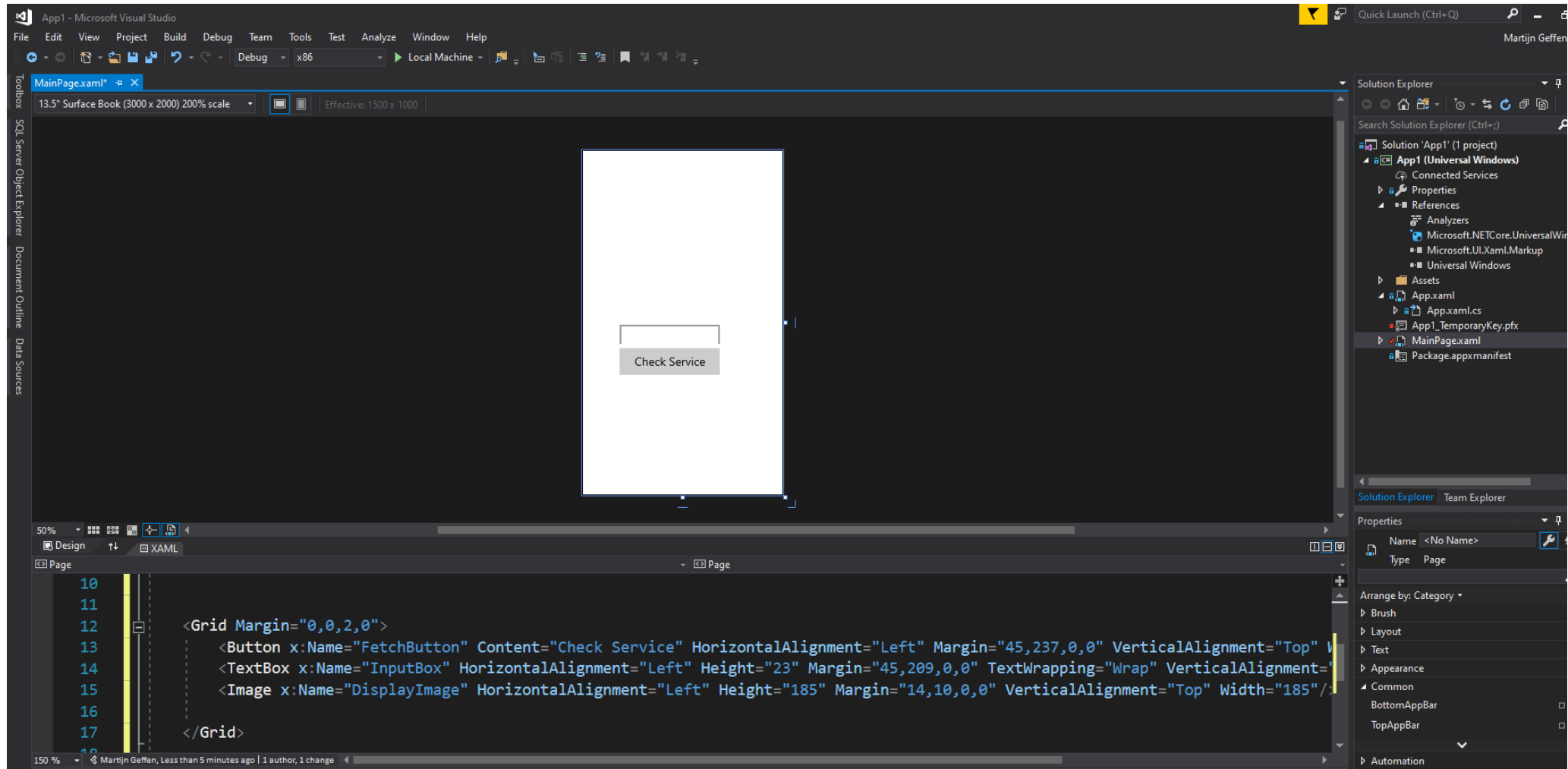
Demonstration

Building GUIs with WPF in
PowerShell Code



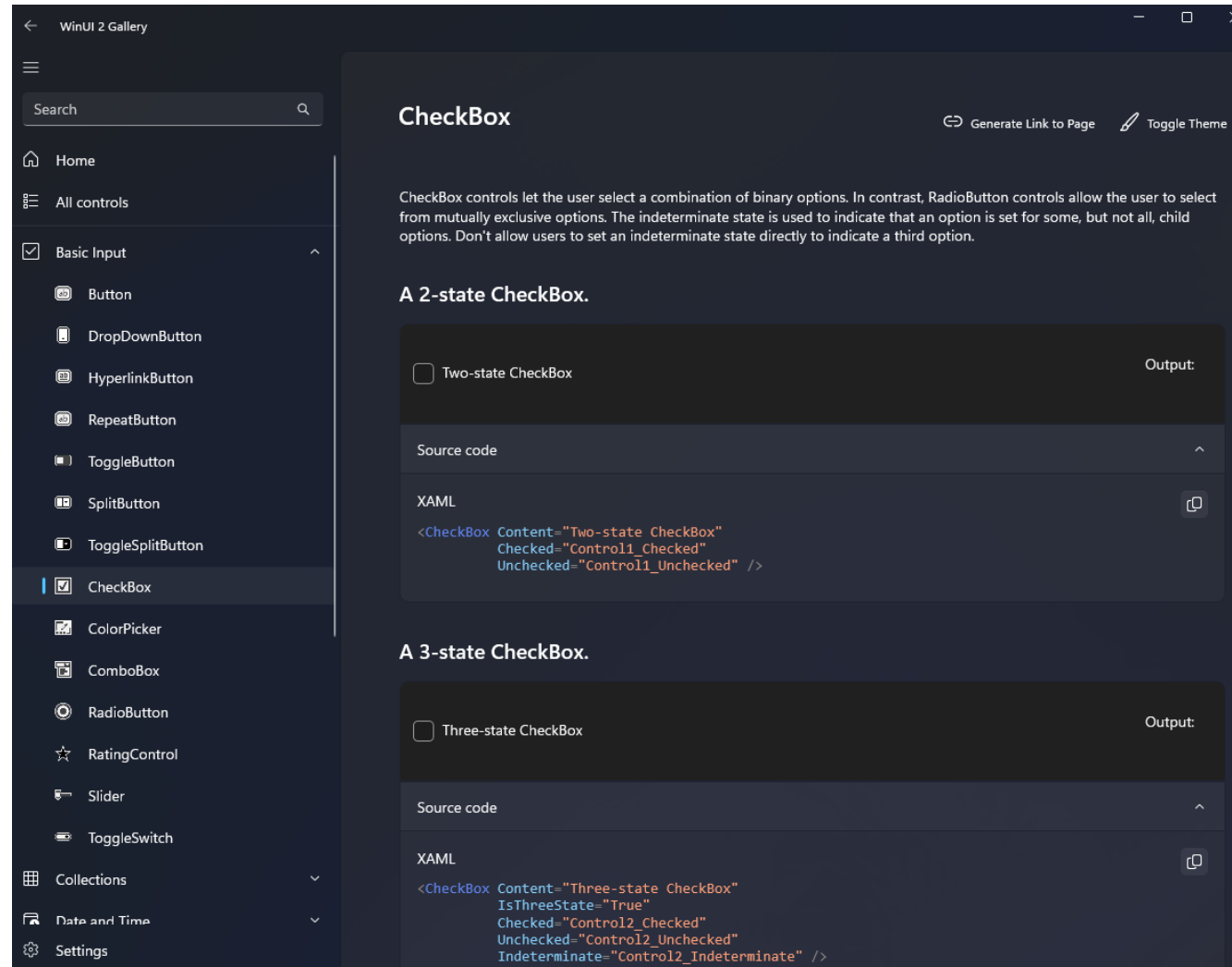
Tools to create XAML code (continued)

Visual Studio with C# WPF template



Tools to create XAML code

WinUI 2 Gallery



The screenshot displays the WinUI 2 Gallery application interface. On the left is a dark sidebar with a search bar and a list of control categories: Home, All controls, Basic Input (expanded), ColorPicker, ComboBox, RadioButton, RatingControl, Slider, ToggleSwitch, Collections, Date and Time, and Settings. The 'Basic Input' category is expanded, showing a list of controls including Button, DropDownButton, HyperlinkButton, RepeatButton, ToggleButton, SplitButton, ToggleSplitButton, and the selected 'CheckBox'. The main content area is titled 'CheckBox' and includes a description: 'CheckBox controls let the user select a combination of binary options. In contrast, RadioButton controls allow the user to select from mutually exclusive options. The indeterminate state is used to indicate that an option is set for some, but not all, child options. Don't allow users to set an indeterminate state directly to indicate a third option.' Below the description, there are two sections: 'A 2-state CheckBox.' and 'A 3-state CheckBox.'. Each section contains a visual representation of the control (a checkbox), its 'Output' label, a 'Source code' section with a collapse arrow, and a 'XAML' section with a copy icon. The XAML code for the 2-state checkbox is:

```
<CheckBox Content="Two-state CheckBox" Checked="Control1_Checked" Unchecked="Control1_Unchecked" />
```

 The XAML code for the 3-state checkbox is:

```
<CheckBox Content="Three-state CheckBox" IsThreeState="True" Checked="Control2_Checked" Unchecked="Control2_Unchecked" Indeterminate="Control2_Indeterminate" />
```

WinUI 2 Gallery

Search

Home

All controls

Basic Input

Button

DropDownButton

HyperlinkButton

RepeatButton

ToggleButton

SplitButton

ToggleSplitButton

CheckBox

ColorPicker

ComboBox

RadioButton

RatingControl

Slider

ToggleSwitch

Collections

Date and Time

Settings

CheckBox

Generate Link to Page Toggle Theme

CheckBox controls let the user select a combination of binary options. In contrast, RadioButton controls allow the user to select from mutually exclusive options. The indeterminate state is used to indicate that an option is set for some, but not all, child options. Don't allow users to set an indeterminate state directly to indicate a third option.

A 2-state CheckBox.

☐ Two-state CheckBox Output:

Source code

XAML

```
<CheckBox Content="Two-state CheckBox"
Checked="Control1_Checked"
Unchecked="Control1_Unchecked" />
```

A 3-state CheckBox.

☐ Three-state CheckBox Output:

Source code

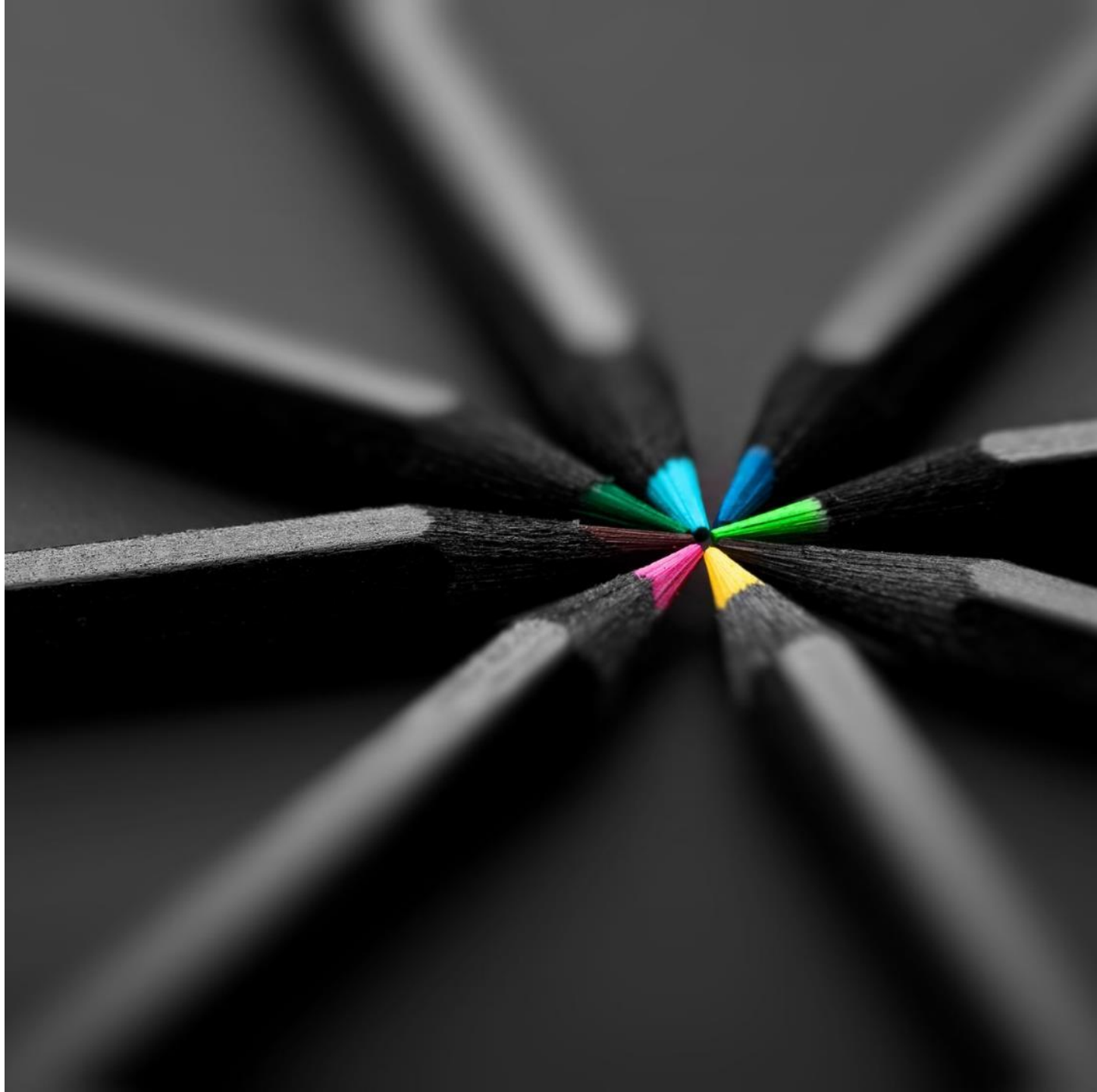
XAML

```
<CheckBox Content="Three-state CheckBox"
IsThreeState="True"
Checked="Control2_Checked"
Unchecked="Control2_Unchecked"
Indeterminate="Control2_Indeterminate" />
```

Questions?



Introduction to Visual Studio



Why Use Visual Studio for XAML?



Powerful IDE for writing XAML or C# based GUI code



GUI for editing XAML code in real time

Changing an option in the rendered form causes VS to update the code and the opposite



Drag and drop functionality for adding new controls



Easy GUI guided based support for setting advanced options

Visual Studio Editions



Visual Studio Express

Free for everyone but discontinued



Visual Studio Community

Free for use on Opensource, Classroom training, Academic research

Not for Enterprise



Visual Studio Professional

Same version as community but for Enterprise



Visual Studio Enterprise

Same version as Professional but with extra tools (Mainly live testing and debugging)

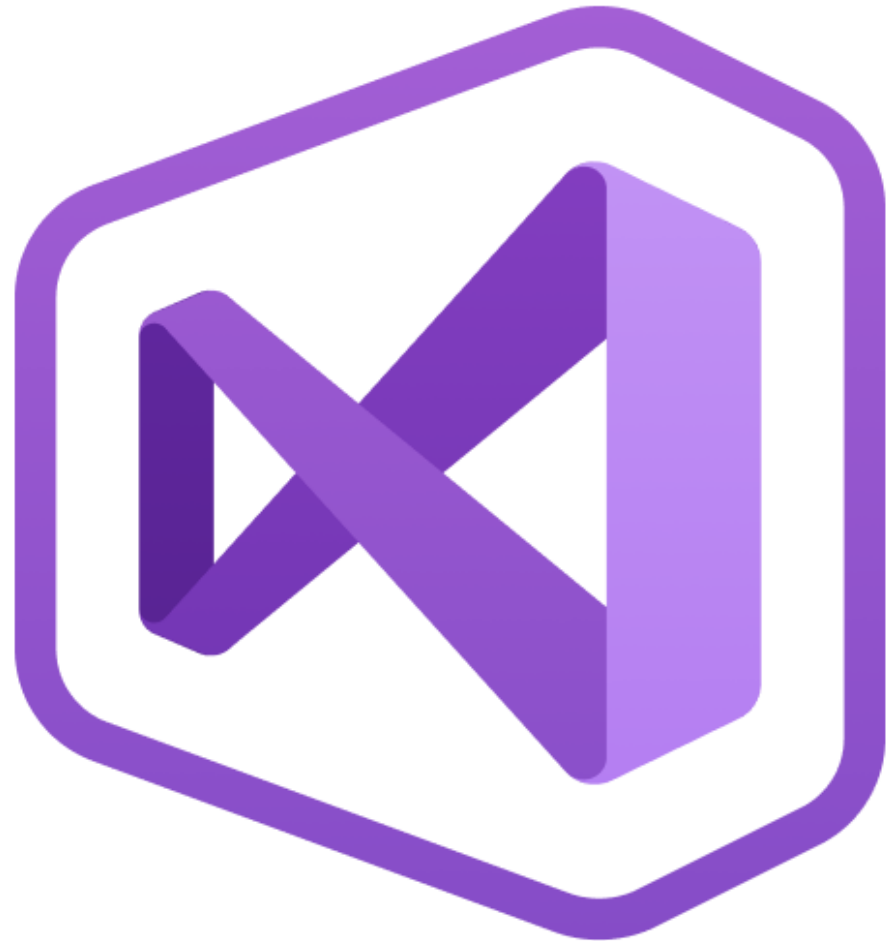
Visual Studio Versions

Currently in support major versions:

- Visual Studio 2019
- Visual Studio 2022

Extended support major versions:

- Visual Studio 2017
- Visual Studio 2015



VS Installation for XAML Requirements

OS and system requirements:

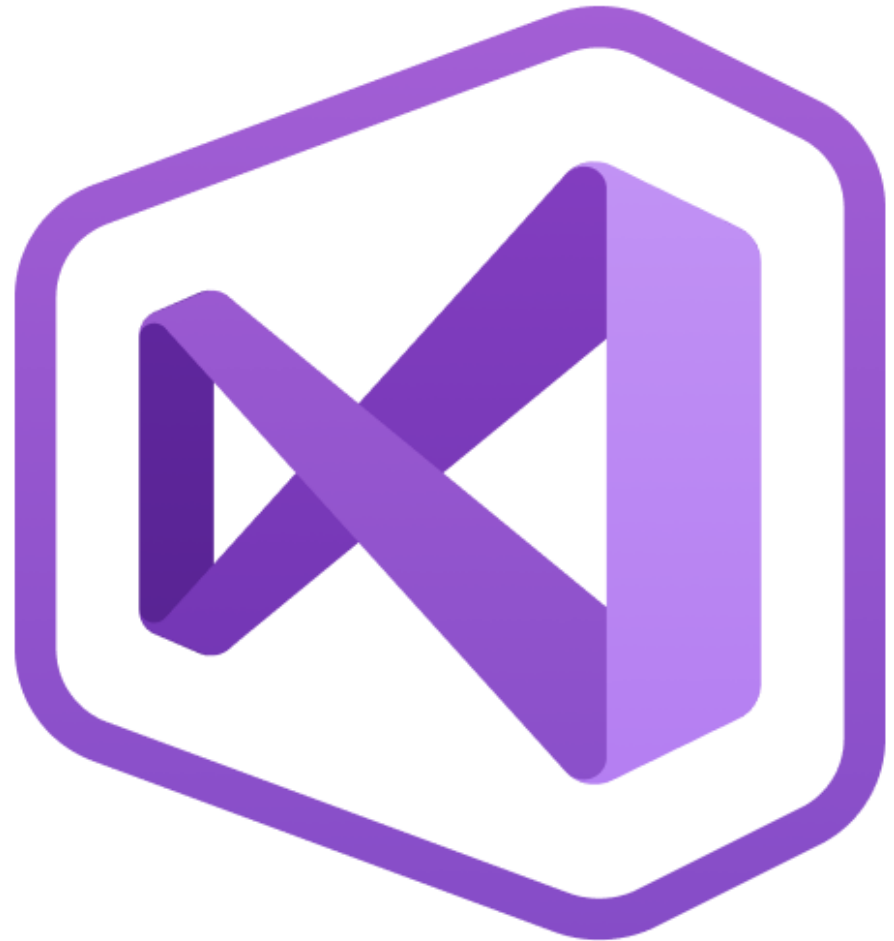
- Windows 10 or newer
- .NET Version 3.5+

Visual studio options:

- .NET desktop development feature

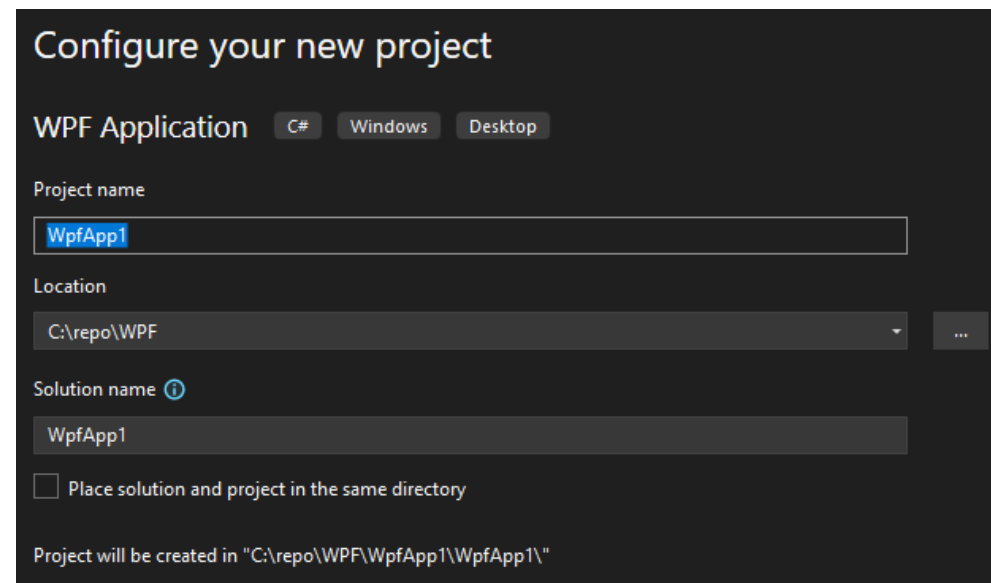
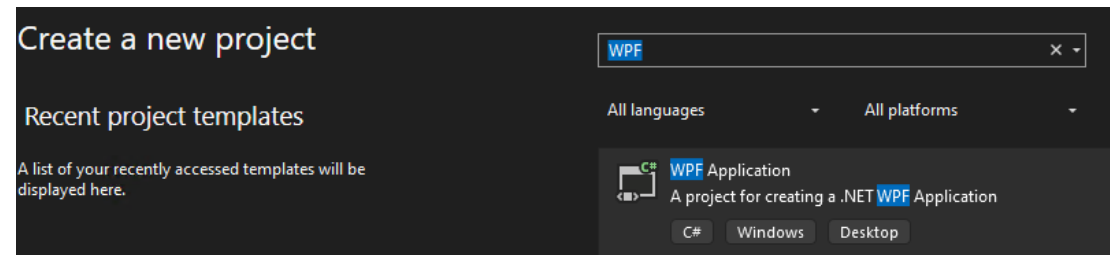
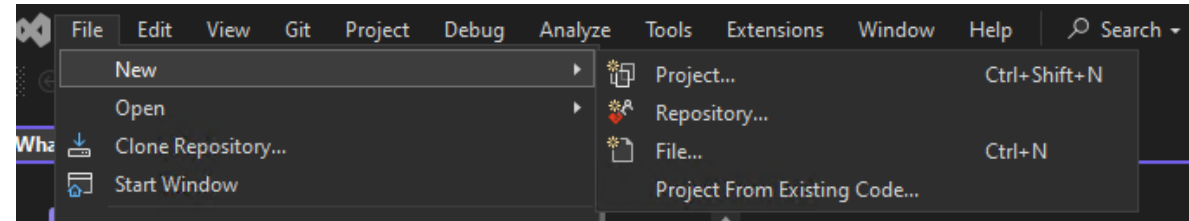
Custom XAML form controls:

- Visual Studio Blend



Creating a XAML project

- File
- New
- Project
- Search for WPF
- Select WPF Application
- Project Name
- Location
- OK



Visual Studio Layout

Solution Explorer

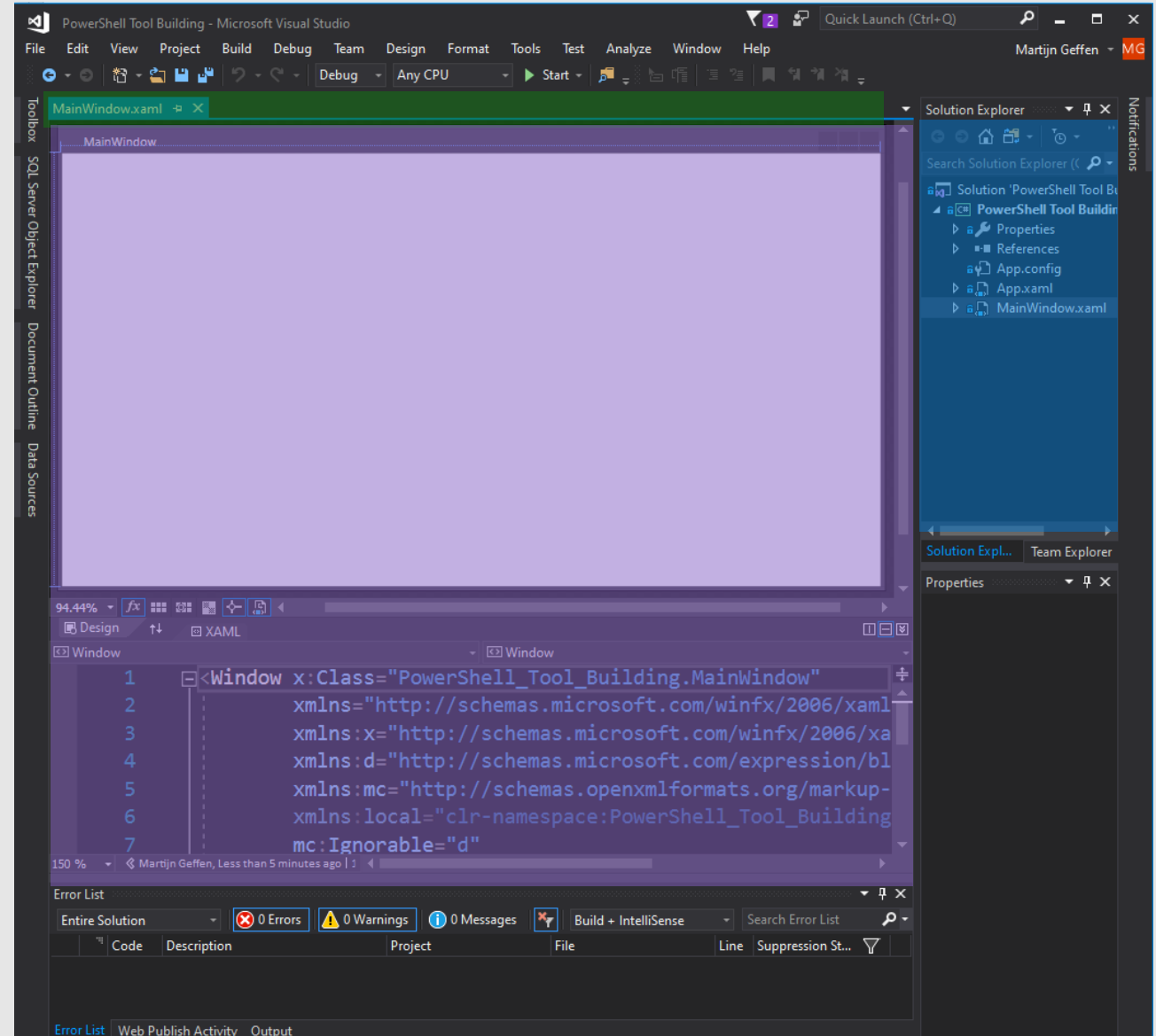
- Open files in the solution
- Focus on MainWindow.xaml

Tabs

- Files that are open
- Focus on MainWindow.xaml

Viewport

- GUI with code behind
- Code only



Visual Studio Layout (continued)

Flyout

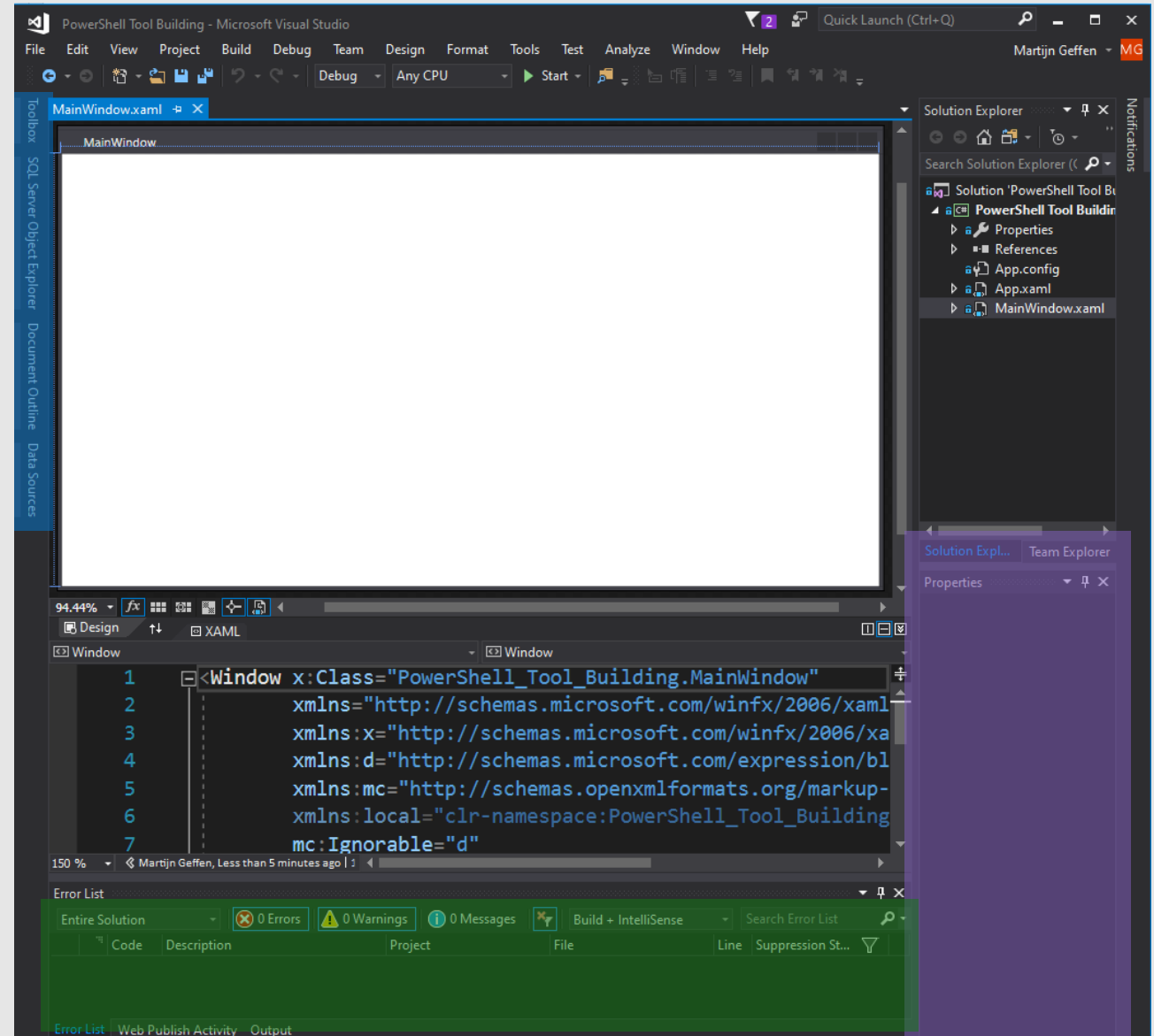
- WPF controls
- Data connectors

Properties & Events

- Selected object properties
- Event triggers

Error List

- Current solution Error and Warnings
- Bing search Remediation



Demonstration

Visual Studio



Questions?



Graphical User Interface Design



WPF GUI design

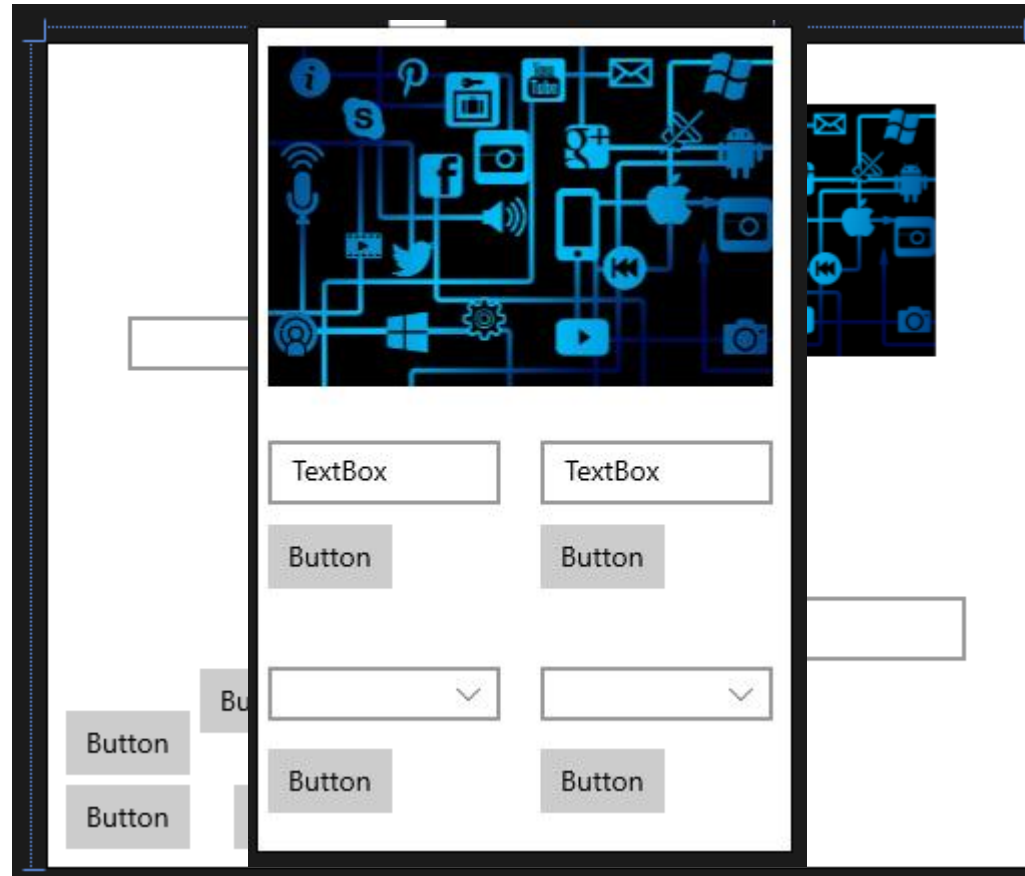
What makes a good User Interface ??

Minimize:

- Input fields
- Amount of buttons

Tabs:

- Help with large amounts of data



Scale:

- Scale background
- Scale images
- Scale input boxes

Align:

- Proper Align all elements
- Natural feeling

XAML Controls

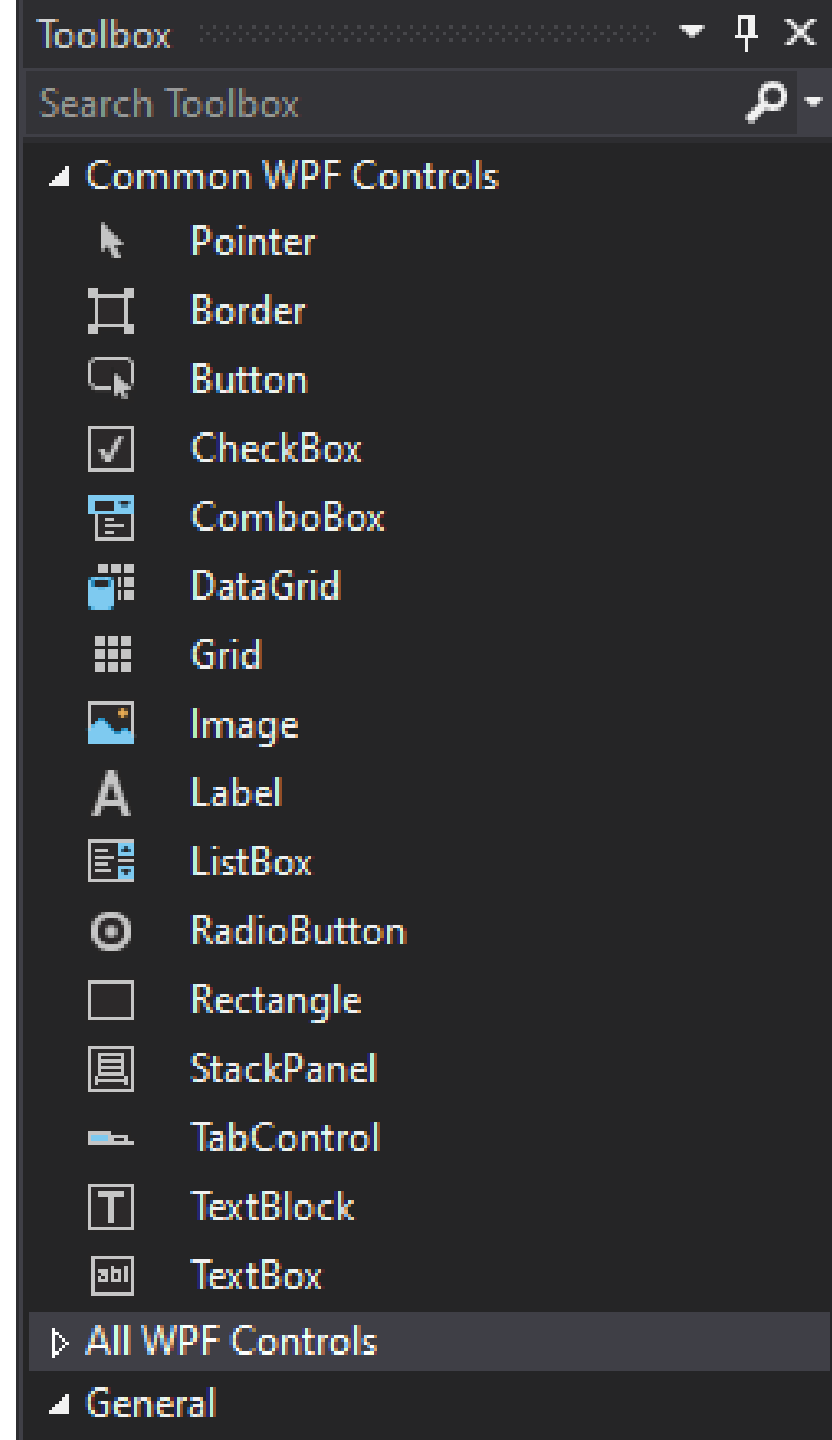
A Control is a predefined GUI behavior component

Controls can be found in the toolbox within Visual Studio

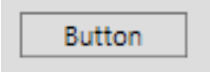
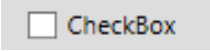
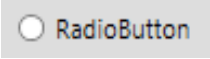
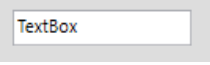
Visual studio comes with approximately 50 controls

Free app to try controls and view code samples

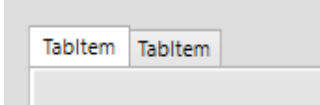
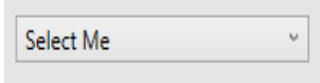
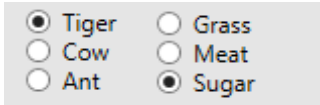
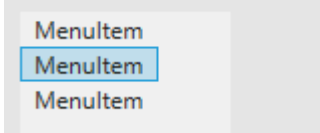
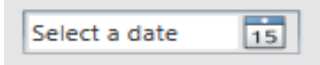
- [WinUI 2 Gallery - Free download and install on Windows | Microsoft Store](#)
- [GitHub - microsoft/WinUI-Gallery:](#)



Basic GUI controls

Element	Use Case	Visual Control	XAML Sample
Button	Pressed to trigger event		<code><Button Content="Button" /></code>
Checkbox	Enable disable an option		<code><CheckBox Content="CheckBox" /></code>
Radiobutton	Selecting a single option from multiple choice keep all options visible		<code><RadioButton Content="RadioButton" /></code>
TextBox	Provide user with input field Can be prepopulated		<code><TextBox Text="TextBox"/></code>
PasswordBox	Input field with hidden characters		<code><PasswordBox Password="we1123"/></code>
Label	Visible Comment on a control Explanation for end user		<code><Label Content="Label" /></code>

Advanced / Grouping GUI controls

Element	Use Case	Visual Control	XAML Sample
TabControl	Providing multiple tabs to group actions or views together		<pre><TabControl> <TabItem Header="Tab1"> </TabItem> </TabControl></pre>
ComboBox	Selecting a single option from multiple choice Make all options visible when selecting and hide again on select		<pre><ComboBox width="180"> <ComboBoxItem Content="item1"/> </ComboBox></pre>
StackPanel	Group object together in a stack A stack of radio buttons can have only 1 selected		<pre><StackPanel> <RadioButton Content="Tiger"/> </StackPanel></pre>
Menu	A menu with event driven triggers when clicked upon		<pre><Menu> <MenuItem Header="MenuItem"/> </Menu></pre>
DatePicker	Allows to select a date out of a calendar control that displays when clicking the date icon		<pre><DatePicker/></pre>

Demonstration

Gui Controls using **winui 2**
Gallery

[WebUI 2 Download](#)

[WPF Galery Preview](#)

[WinUI Gallery GitHub
Repository](#)



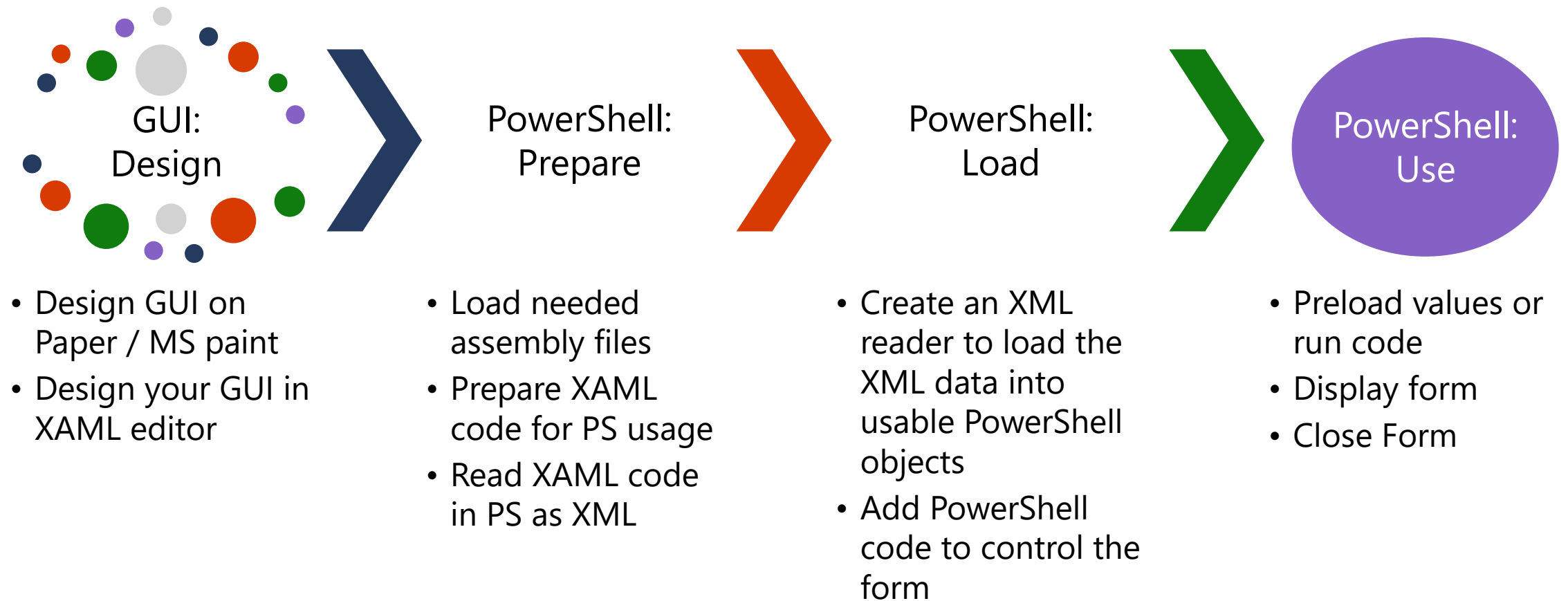
Questions?



Your first GUI



WPF / XAML GUI Workflow



XAML GUI for PowerShell

Name all controls and windows in the GUI

This will be used by PowerShell to create variables (Explained later)

Do not create any code behind the form (*.xaml.cs file)

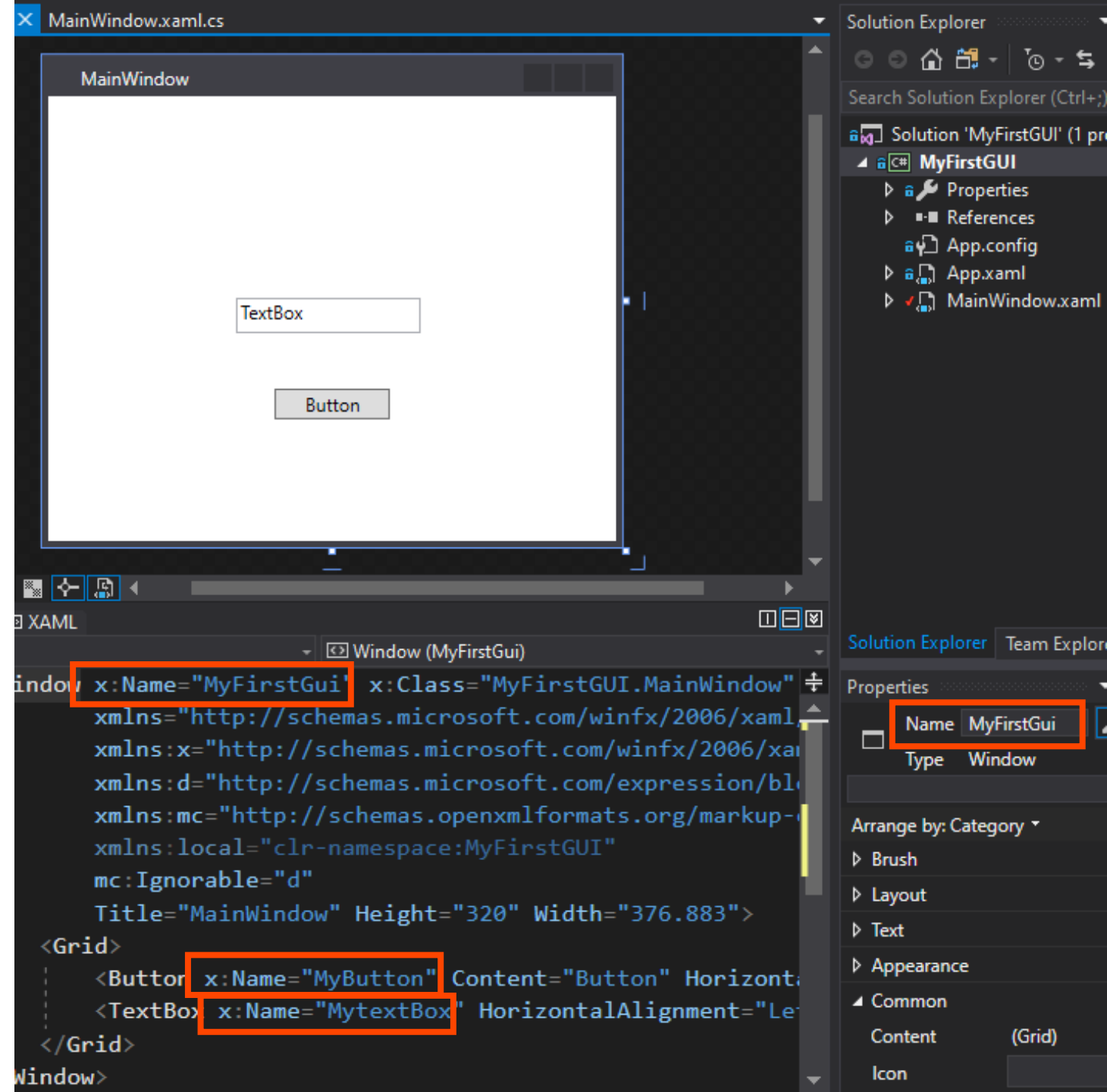
This is the code that needs to be created in PowerShell

3rd party controls need to be added via PowerShell

This complicates the GUI and requires dependencies

**During migration from VS to PowerShell, only use the
*.xaml formatting code**

My First GUI - Sample



Preparing the XAML Code for PowerShell

VS adds headers not supported by Windows.Markup.XamlReader

```
#Add WPF assemblies
Add-Type -AssemblyName PresentationCore,PresentationFramework

#Cleanup XAML options
$XAML= [XML](Get-Content -Path '$PSScriptRoot\MainWindow.xaml' -Raw)
$XAML.window.RemoveAttribute('x:Class')
$XAML.window.RemoveAttribute('xmlns:local')
$XAML.window.RemoveAttribute('xmlns:d')
$XAML.window.RemoveAttribute('xmlns:mc')
$XAML.window.RemoveAttribute('mc:Ignorable')

#read XML as XAML
$XAMLreader = New-Object System.Xml.XmlNodeReader $XAML
$Rawform = [Windows.Markup.XamlReader]::Load($XAMLreader)
```

Converting GUI Form to Hash Table

Single hash table provides easy processing of PS code in code behind GUI

```
#add XML namespace manager
$xmlNamespaceManager =
[System.Xml.XmlNamespaceManager]::New($XAML.NameTable)
$xmlNamespaceManager.AddNamespace('x', 'http://schemas.microsoft.com/wi
nfx/2006/xaml')

#Create hash table containing a representation of all controls
$GUI = @{}
$namedNodes = $XAML.SelectNodes("//*[@x:Name]", $xmlNamespaceManager)
$namedNodes | ForEach-Object -Process {$GUI.Add($_.Name,
$Rawform.FindName($_.Name))}

#Show GUI
$Rawform.ShowDialog()
```

Demonstration

One Hash to Rule Them All:
PowerShell GUI Control
Management



Questions?



[illegible]

PowerShell as an Engine For WPF



**PowerShell can
be used as the
engine behind
the form**



**Well
documented
MSDN for all
controls**



**All form code
should be
written before
you display the
form**



Advantage:

- No need to learn other language to Power great visual forms



Disadvantage:

- No debugger available (Shell access lockout while form is visible)
- PowerShell has no native form refresh as an asynchronous task

Control Properties

Controls have Properties you can change / update

Use **Get-member** to view Property types

```
PS C:\> $Gui.Inputtext.gettype().fullname  
> System.Windows.Controls.TextBox
```

```
PS C:\> $Gui.Inputtext | Get-Member -Type Property -name *Text*
```

> Name	MemberType	Definition
> ----	-----	-----
> ContextMenu	Property	System.Windows.Controls.ContextMenu ...
> DataContext	Property	System.Object DataContext {get;set;}
> SelectedText	Property	string SelectedText {get;set;}
> SelectionTextBrush	Property	System.Windows.Media.Brush Selection...
> Text	Property	string Text {get;set;}
> TextAlignment	Property	System.Windows.TextAlignment TextAli...
> TextDecorations	Property	System.Windows.TextDecorationCollect...
> TextWrapping	Property	System.Windows.TextWrapping TextWrap...

Programming Control Properties

Define properties before the form loads

Changing properties without form refresh wont show

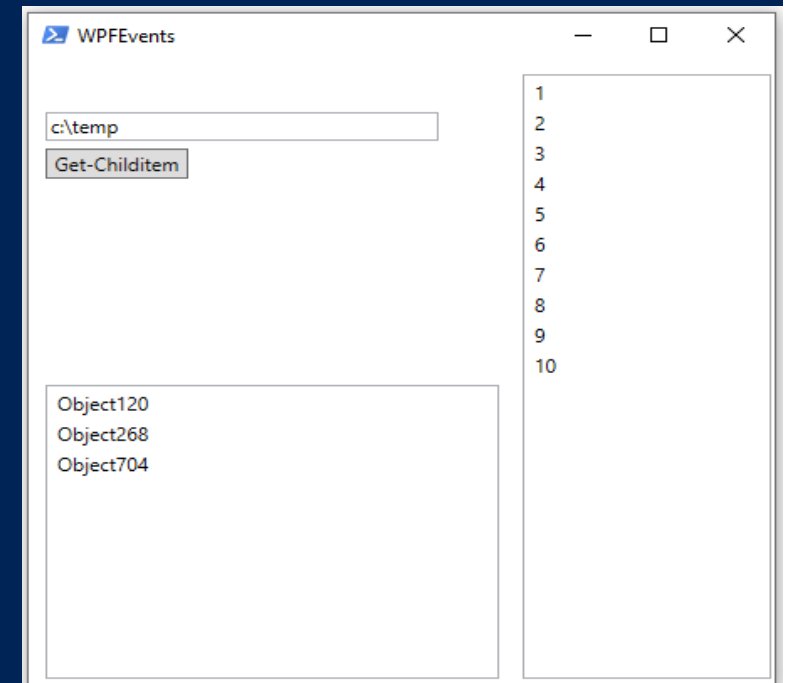
Most events will trigger form refresh

```
PS C:\> $GUI.Inputtext.Text = "c:\temp"
```

```
PS C:\> 1..10 | %{$gui.Itemlist.Items.add($_)}
```

```
PS C:\> $1 = "Object120","Object268","Object704"
```

```
PS C:\> $1 | %{$gui.Detaileditemlist.Items.add($_)}
```



Event Driven Actions

Controls have events that can be used as triggers for code



Use **Get-Member** to view event types

```
PS C:\> $Gui.Get_childitem.gettype().fullname  
> System.Windows.Controls.Button
```

```
PS C:\> ($Gui.Get_childitem | Get-Member -Type event).count  
> 111
```

```
PS C:\> $Gui.Get_childitem | Get-Member -Type event -name *click*
```

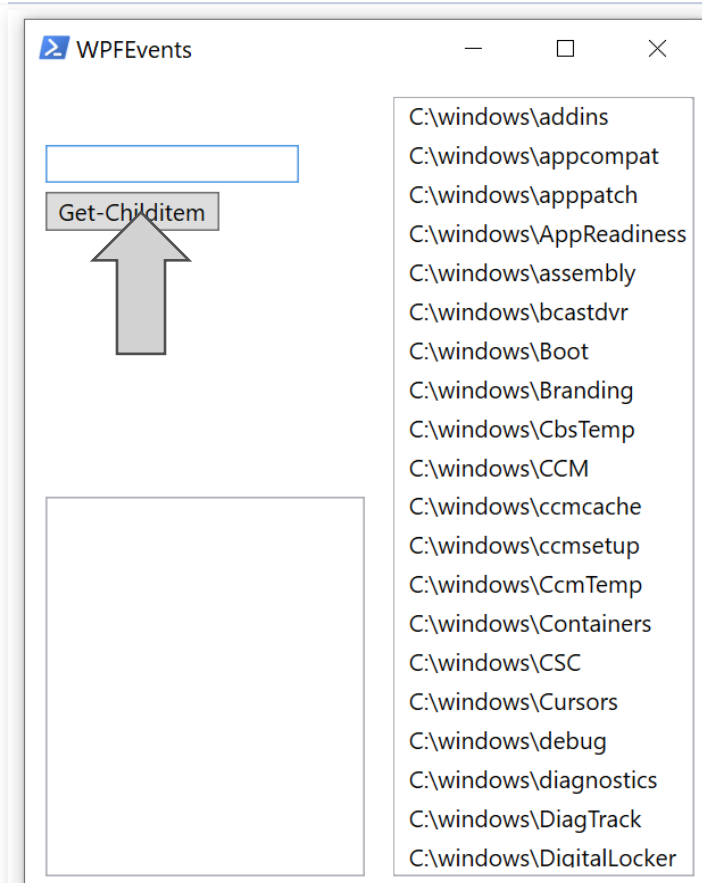
> Name	MemberType	Definition
> ----	-----	-----
> Click	Event	System.Windows.RoutedEventHan...
> MouseDoubleClick	Event	System.Windows.Input.MouseBut...
> PreviewMouseDoubleClick	Event	System.Windows.Input.MouseBut...

Programming Events

Event triggers are added to code using "**Add_Eventname**({Code})"

The Form is unresponsive during PowerShell code execution

```
$Gui.Get_Childitem.Add_Click({  
    If (!([string]::IsNullOrEmpty($Gui.Inputtext.text)))  
    {  
        $childitems = Get-ChildItem -Path $Gui.Inputtext.text  
        foreach ($item in $childitems)  
        {  
            $Gui.Itemlist.Items.Add($item.fullname)  
        }  
        $Gui.Inputtext.clear()  
    }  
})
```



Demonstration

Form unresponsiveness



Data Binding

A functionality to update a control when the source it points to changes

- Updating source can cause a chain of other controls to get updated automatically

Ways to declare data binding

- Directly in XAML referencing an other control as source
- In PowerShell code referencing a PowerShell variable object

A data bind can be used by all children of the parent

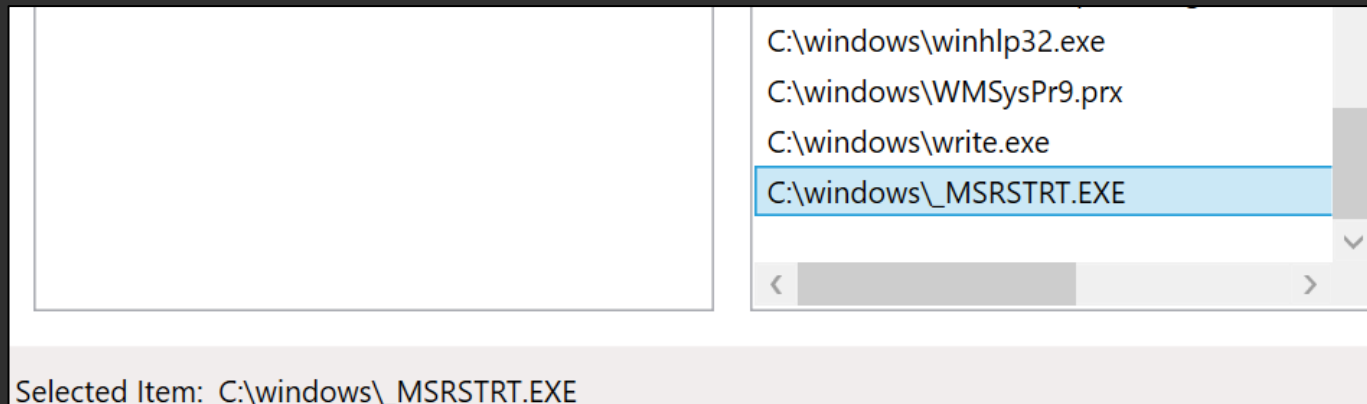
XAML Based Data Binding

Declared in XAML code

Only works on form refresh

Anything selected in "Itemlist" will be displayed in the text property of "Itemlistdisplay"

```
<ListBox x:Name="Itemlist" Width="250" AllowDrop="True"/>  
<TextBlock x:Name="Itemlistdisplay" TextWrapping="Wrap"  
Text="{Binding ElementName=Itemlist, Path=SelectedItem, Mode=OneWay }"/>
```



PowerShell Based Data Binding

Declared within PowerShell code

Displays on form refresh or via Asynchronous process

```
# Create a datacontext for the textbox and set it
$DataContext = New-Object
System.Collections.ObjectModel.ObservableCollection[Object]
$DataContext.Add((get-date -f HH:mm:ss))
$Gui.timebox.DataContext = $DataContext

# Create and set a binding on the textbox object
$Binding = New-Object System.Windows.Data.Binding
$Binding.Path = "[0]"
$Binding.Mode = [System.Windows.Data.BindingMode]::OneWay
[void][System.Windows.Data.BindingOperations]::SetBinding($Gui.timebox,
[System.Windows.Controls.TextBox]::TextProperty, $Binding)
```

Demonstration

Data Binding



Questions?



Advanced Techniques



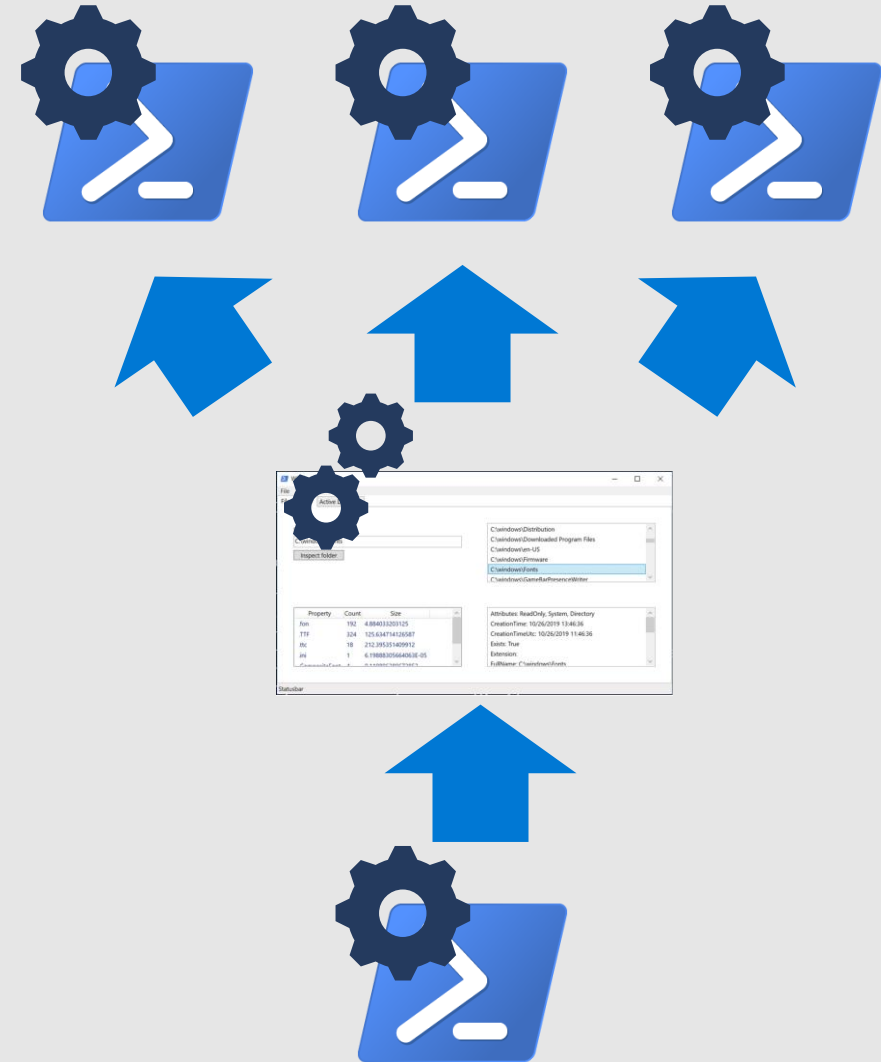
Why Does My GUI Get Unresponsive

XAML in PowerShell runs by default on the PS thread

PS is either updating the form or running the code behind

Techniques to overcome GUI locks due to running code:

- Creating timers
- Use Background workers
- Use Runspaces
- Mix of the above



Creating Timers



Easy to implement



Run in a dispatcher queue



Timer guarantees only interval has elapsed

Exact start time is not guaranteed



Used for short and fast running code

Maximum of ~2 sec runtime

Creating Timers - Example

```
$rawform.add_sourceinitialized({  
    #create timer object  
    $timer = new-object System.Windows.Threading.DispatcherTimer  
    $timer.interval = [timespan]"0:0:1"  
  
    #add event per tick  
    $timer.add_tick({  
        $DataContext[0] = get-date -f HH:mm:ss  
    })  
  
    $timer.start()  
})
```

Use a Runspace for GUI

- More code overhead needed to synchronize data between threads
- Runs in a separate .Net Runspace
 - GUI will not lock up as it has its own thread
- Used to isolate the GUI processing from PowerShell code
 - Can be used to run single, longer running processes while GUI remains responsive
- PowerShell console remains responsive for additional input
- Used to initialize Form or run data updates in background

Use a Runspace to Run WFP GUI - Example

```
$Rawform = [hashtable]::Synchronized(@{})  
$GUI = [hashtable]::Synchronized(@{})  
$GUIRS = [hashtable]::Synchronized(@{})  
  
$GUIRS.psscriptroot = $PSScriptRoot  
$GUIRS.runspace = [RunspaceFactory]::CreateRunspace()  
$GUIRS.runspace.ApartmentState = "STA"  
$GUIRS.runspace.ThreadOptions = "ReuseThread"  
$GUIRS.runspace.Open()  
$GUIRS.runspace.SessionStateProxy.SetVariable("Rawform", $Rawform)  
$GUIRS.runspace.SessionStateProxy.SetVariable("GUIRS", $GUIRS)  
$GUIRS.runspace.SessionStateProxy.SetVariable("GUI", $GUI)  
$GUIRS.psCmd.Runspace = $GUIRS.runspace  
  
$GUIRS.Handle = $GUIRS.psCmd.AddScript({.....}).BeginInvoke()
```

Start Runspace
Async

WPF code goes here

Questions?



PowerShell Graphical User Interfaces

